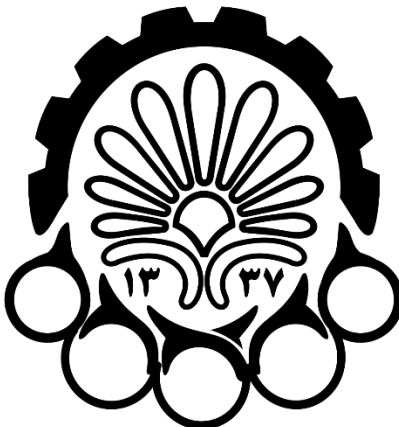


بسمه تعالی



دانشگاه صنعتی امیر کبیر
(پلی تکنیک تهران)

موضوع :

گزارش تمرین چهارم درس یادگیری ژرف

دانشجو : متین قاسمی

شماره دانشجویی : 402112098

استاد : دکتر مصطفی شمسی – دکتر زاهد رحمتی

بهار ۱۴۰۳

سوال 1 :

در این بخش خواسته شده است مجموعه ای از ویژگی های گراف را چاپ کنیم. برای اینکار از کتابخانه networkx استفاده شده است. همچنین برای ابعاد بردار ویژگی های راس و یال فرض کردم به صورت یک بردار به عنوان پارامتر در راس و یال قرار دارد همچنین اگر ابعاد آن برای راس یا یال های مختلف تفاوت داشته باشد همه ی ابعاد را نشان میدهد. بقیه بخش ها صرفا از متود های کتابخانه استفاده شده است.

```
def print_graph_stat(graph):
    # Number of nodes
    print(f'Number of nodes: {graph.number_of_nodes()}')

    # Number of edges
    print(f'Number of edges: {graph.number_of_edges()}')

    # Node features (assuming node features are stored as attributes in each node)
    node_features = list(graph.nodes(data=True))
    if node_features and any('features' in data for _, data in node_features):
        feature_dims = set(len(data['features']) for _, data in node_features if 'features' in data)
        print(f"Graph has node features with dimensions: {feature_dims}")
    else:
        print("Graph has no node features")

    # Edge features (assuming edge features are stored as attributes in each edge)
    edge_features = list(graph.edges(data=True))
    if edge_features and any('features' in data for _, _, data in edge_features):
        edge_feature_dims = set(len(data['features']) for _, _, data in edge_features if 'features' in data)
        print(f"Graph has edge features with dimensions: {edge_feature_dims}")
    else:
        print("Graph has no edge features")

    # Isolated nodes
    print(f'Number of isolated nodes: {nx.number_of_isolates(graph)}')

    # Self-loops
    print(f'Number of self-loops: {nx.number_of_selfloops(graph)}')

    print(f'Graph is {"directed" if graph.is_directed() else "not directed"}')

print_graph_stat(networkx_data)
# or print_graph_stat(pyg_data)
```

همچنین خروجی تابع برای گراف داده شده به صورت زیر است. که مشخصات گراف را نشان میدهد.

```
Number of nodes: 2708
Number of edges: 5278
Graph has no node features
Graph has no edge features
Number of isolated nodes: 0
Number of self-loops: 0
Graph is not directed
```

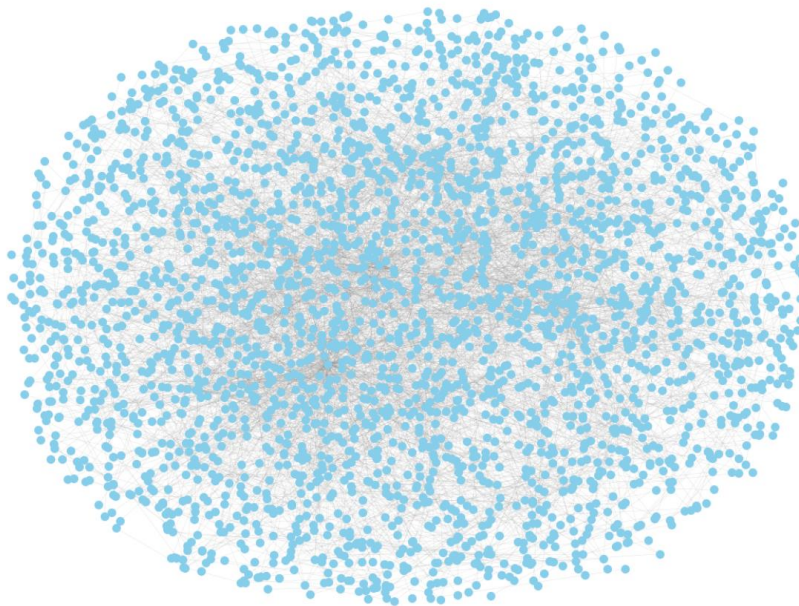
سوال 2 :

در این بخش باید تابعی نوشته میشد که گراف را نشان دهد. برای اینکار ابعاد راس و یال ها به گونه ای در نظر گرفته شده اند که بتواند گراف بزرگ را نمایش دهد.

```
def visualize_graph(graph):  
    # Generate positions for all nodes using the spring layout  
    pos = nx.spring_layout(graph, k=0.2)  
  
    # Create a figure  
    plt.figure(figsize=(16, 12))  
  
    nx.draw_networkx_nodes(graph, pos, node_size=50, node_color='skyblue')  
    nx.draw_networkx_edges(graph, pos, width=0.1, edge_color='gray', alpha=0.7)  
  
    # Set title and remove axes  
    plt.title('Graph Visualization')  
    plt.axis('off')  
  
    # Show the plot  
    plt.show()  
  
visualize_graph(networkx_data)
```

شکل گراف مورد نظر نیز به صورت زیر است.

Graph Visualization



سوال 3 :

در این بخش ویژگی های گراف در سطح راس محاسبه شده است. یک دیکشنری درنظر گرفته شده است که به ازای هر راس ویژگی های مورد نظر را در خود ذخیره میکند و در نهایت این دیکشنری چاپ میشود تا کل گراف نمایش داده شود.

```
def node_level_feature_graph(graph):
    nodes_features = {node: {} for node in list(graph.nodes)}
    # List of degrees and its average
    graph_degree = graph.degree()
    avg_degree = np.mean([d for _, d in graph_degree])
    for node, degree in graph_degree:
        nodes_features[node]['degree'] = degree

    # List of clustering coefficients and its average
    graph_clustering_coeffs = nx.clustering(graph)
    avg_clustering_coeffs = np.mean(list(graph_clustering_coeffs.values()))
    for node, value in graph_clustering_coeffs.items():
        nodes_features[node]['clustering_coeffs'] = round(value, 3)

    # List of closeness centralities and its average
    graph_closeness_centrality = nx.closeness centrality(graph)
    avg_closeness_centrality = np.mean(list(graph_closeness_centrality.values()))
    for node, value in graph_closeness_centrality.items():
        nodes_features[node]['closeness_centrality'] = round(value, 3)

    # List of eigenvector centralities and its average
    graph_eigenvector_centrality = nx.eigenvector centrality(graph)
    avg_eigenvector_centrality = np.mean(list(graph_eigenvector_centrality.values()))
    for node, value in graph_eigenvector_centrality.items():
        nodes_features[node]['eigenvector_centrality'] = round(value, 3)

    # List of betweenness centralities and its average
    graph_betweenness_centrality = nx.betweenness centrality(graph)
    avg_betweenness_centrality = np.mean(list(graph_betweenness_centrality.values()))
    for node, value in graph_betweenness_centrality.items():
        nodes_features[node]['betweenness_centrality'] = round(value, 3)
```

خروجی تابع به شکل زیر است. که مقادیر خواسته شده را نشان میدهد.

```
Average degree: 3.8981
Average clustering coefficient: 0.2407
Average closeness centrality: 0.1375
Average eigenvector centrality: 0.0048
Average betweenness centrality: 0.0017
```

Node	Degree	Clustering	Closeness	Eigenvector	Betweenness
0	3	0.333	0.144	0.0	0.0
1	3	0	0.151	0.0	0.001
2	5	0	0.179	0.001	0.004
3	1	0	0.0	0.0	0.0
4	5	0.7	0.153	0.005	0.001
5	3	0.333	0.13	0.001	0.0
6	4	0.333	0.155	0.0	0.001
7	1	0	0.0	0.0	0.0
8	3	0	0.159	0.0	0.003
9	2	1.0	0.13	0.0	0.0
10	2	0	0.16	0.0	0.001
11	2	1.0	0.158	0.0	0.0
12	4	0.5	0.001	0.0	0.0
13	2	0	0.176	0.001	0.001

درجه راس تعداد یال های وصل شده به آن را نشان میدهد. بنابراین راس های با درجه بالاتر مرکزی تر هستند و تاثیر زیادی روی گراف دارند و به نوعی دارای اهمیت هستند.

ضریب خوشگی (Clustering Coefficient) داره میزان نزدیک بودن یک راس با همسایه هاش به گراف کامل رو نشون میده به نوعی میزان ارتباط همسایه های یک راس به یکدیگر رو میسنجه بنابراین بالا بودن این شاخص میتواند نشان دهنده ی وجود گروه های کوچک در گراف است.

معیار Closeness Centrality نشان دهنده نزدیک بودن یک راس با بقیه ی راس های گراف است و با استفاده از کوتاه ترین مسیر بین راس های محاسبه میشود. بالا بودن این معیار نشان دهنده این است که یک راس چقدر سریع با دیگر راس ها در ارتباط است و میتواند اطلاعات را جابه جا کند.

معیار Eigenvector Centrality میان تاثیر هر راس را از میزان تاثیرگذاری راس های همسایه اش در نظر میگیرد. بالا بودن این معیار نه تنها اهمیت خود راس بلکه تاثیرگذاری همسایه های آن را نیز نشان میدهد.

در نهایت معیار Betweenness Centrality نشان میدهد چقدر یک راس در مسیر کوتاه های مسیر بین راس ها قرار دارد. راس هایی که این معیار در آن ها بالا است میتوانند بر مسیر حرکت اطلاعات کنترل داشته باشند.

موارد بالا را اگر برای راس ۴ در نظر بگیریم متوجه میشیم درجه ی بالاتر از میانگین دارد پس یک راس مرکزی است همچنین ضریب خوشگی بالای آن نشان میدهد که در یک گروه کوچک قرار دارد که همسایه های آن با هم در ارتباط هستند همچنین Closeness بالاتر از میانگین دارد که باز نشان از اهمیت این راس دارد. مقدار Eigenvector نیز نزدیک به میانگین و کمی بالاتر است که نشان از بی اهمیت نبودن همسایه های آن دارد و مقدار Betweenness آن پایین تر از میانگین است که نشان میدهد در مسیر حرکت اطلاعات نیست اما با توجه به اطلاعات بالا میتوان نتیجه گرفت در یک گروه قرار دارد.

اینکه آیا این اطلاعات برای دسته بندی مدل کافی هستند بستگی به موارد بسیاری دارد اما به صورت کلی علی رغم نشان دادن اطلاعات مفید درباره راس ها اما نمیتوانند ویژگی های پیچیده درون گراف را شناسایی کنند و البته میتوان از این ویژگی های برای دسته بندی استفاده کرد اما انتظار دقت پایین در دسته بندی نسبت به ویژگی های پیچیده تر را خواهیم داشت.

سوال 4 :

در این بخش ویژگی های گراف در سطح یال محاسبه شده است. با توجه به سنگین بودن محاسبات اینکار صرفا برای ۱۰ راس ابتدایی گراف انجام شده است که در صورت نیاز قابل تغییر است.

```
def edge_level_feature_graph(graph):
    nodes = list(graph.nodes())

    print("{:<10} {:<10} {:<15} {:<20} {:<25} {:<20}".format('Node1', 'Node2', 'Shortest path', 'Common neighbors', "Jaccard's coefficient", 'Adamic-Adar index'))
    for node1 in nodes[:10]:
        for node2 in nodes[:10]:
            if node1 != node2:
                # Shortest path distance
                try:
                    shortest_path = nx.shortest_path_length(graph, source=node1, target=node2)
                except nx.NetworkXNoPath:
                    shortest_path = "No path"

                # Common neighbors
                common_neighbors = list(nx.common_neighbors(graph, node1, node2))

                # Jaccard's coefficient
                jaccard_coefficient = nx.jaccard_coefficient(graph, [(node1, node2)])
                jaccard_coefficient_value = next(jaccard_coefficient)[2]

                # Adamic-Adar index
                adamic_adar_index = nx.adamic_adar_index(graph, [(node1, node2)])
                adamic_adar_index_value = next(adamic_adar_index)[2]

                # print(node1, node2, shortest_path, common_neighbors, jaccard_coefficient_value, adamic_adar_index_value)
                print("{:<10} {:<10} {:<15} {:<20} {:<25} {:<20}".format(node1, node2, shortest_path, str(common_neighbors), jaccard_coefficient_value, adamic_adar_index_value))

    edge_level_feature_graph(networkx_data)
```

همچنین خروجی تابع به صورت زیر است.

Node1	Node2	Shortest path	Common neighbors	Jaccard's coefficient	adamic-Adar index
0	1	5	[]	0.0	0
0	2	4	[]	0.0	0
0	3	No path	[]	0.0	0
0	4	7	[]	0.0	0
0	5	8	[]	0.0	0
0	6	8	[]	0.0	0
0	7	No path	[]	0.0	0
0	8	5	[]	0.0	0
0	9	8	[]	0.0	0
1	0	5	[]	0.0	0
1	2	1	[]	0.0	0
1	3	No path	[]	0.0	0
1	4	6	[]	0.0	0
1	5	7	[]	0.0	0
1	6	7	[]	0.0	0
1	7	No path	[]	0.0	0
1	8	4	[]	0.0	0
1	9	7	[]	0.0	0
2	0	4	[]	0.0	0
2	1	1	[]	0.0	0
2	3	No path	[]	0.0	0
2	4	5	[]	0.0	0
2	5	6	[]	0.0	0
2	6	6	[]	0.0	0

کوتاه ترین مسیر (Shortest-Path Distance) بین دو راس نشان دهنده ی فاصله ی آن ها در گراف است که میزان دسترسی دو راس به یکدیگر در شبکه را نشان میدهد.

معیار Common Neighbors میزان شباهت بین همسایه های دو راس را نشان میدهد راس های با همسایه های مشترک بسیار نقش های شبیه به هم در شبکه را ایفا میکنند.

معیار Jaccard's Coefficient یک معیار پیچیده تر نسبت به معیار قبلی است که میزان اشتراک همسایه ها نسبت به اجتماع آن ها را میسنجد و این معیار در دسته بندی به ویژه برای پیشبینی یال مهم است.

معیار Adamic-Adar Index میزان شباهت دو راس را براساس درجه همسایه های مشترک شون مسنجه که همچنان یک معیار مهم برای دسته بندی های مرتبط با یال است.

سوال 5 :

در این سوال باید الگوریتم Weisfeiler-Lehman پیاده سازی میشد. برای اینکه به هر راس یک لیبل (رنگ) دلخواه مانند ۱ دادیم پس در هر مرحله با تجمیع رنگ همسایه ها و سپس هش کردن آن به رنگ آن راس در مرحله بعدی رسیدیم. و در نهایت برای نشان دادن گراف از یک بردار که نشان دهنده ی تکرار رنگ های هش شده در گراف است استفاده کردیم. برای هش کردن هم از روش md5 با استفاده از کتابخانه hashlib استفاده کردیم. همچنین جزییات کد با کامنت ها مشخص است.

```

import hashlib
from collections import Counter

def wl_kernel_graph(graph):
    # Number of iterations for the WL label refinement
    steps = 3

    # Initialize node labels: all nodes start with the same label '1'
    node_labels = {node: '1' for node in graph.nodes}

    # Initialize the set of unique labels (colors)
    set_of_colors = set('1')

    # Perform WL label refinement
    for _ in range(steps):
        new_labels = {}
        # Iterate over each node in the graph
        for node in graph.nodes():
            # Get sorted labels of the node's neighbor
            neighbors = sorted([node_labels[neighbor] for neighbor in graph.neighbors(node)])
            # Create a new label by combining the node's current label with the sorted neighbor labels
            new_label = str(node_labels[node]) + ''.join(neighbors)
            # Hash the combined label to generate a new unique label
            new_label = hashlib.md5(new_label.encode('utf-8')).hexdigest()
            # Store the new label for the node
            new_labels[node] = new_label
            # Add the new label to the set of unique labels (colors)
            set_of_colors.add(new_label)

        # Update node labels for the next iteration
        node_labels = new_labels.copy()

    # Count the frequency of each label in the final iteration
    count_labels = Counter(node_labels.values())
    # Sort the set of unique labels (colors) to create a consistent order
    set_of_colors = sorted(list(set_of_colors))
    # Create the feature vector by counting occurrences of each label in the sorted set of colors
    feature_vector = [count_labels.get(i, 0) for i in set_of_colors]

    return feature_vector

wl_kernel_graph(networkx data)

```